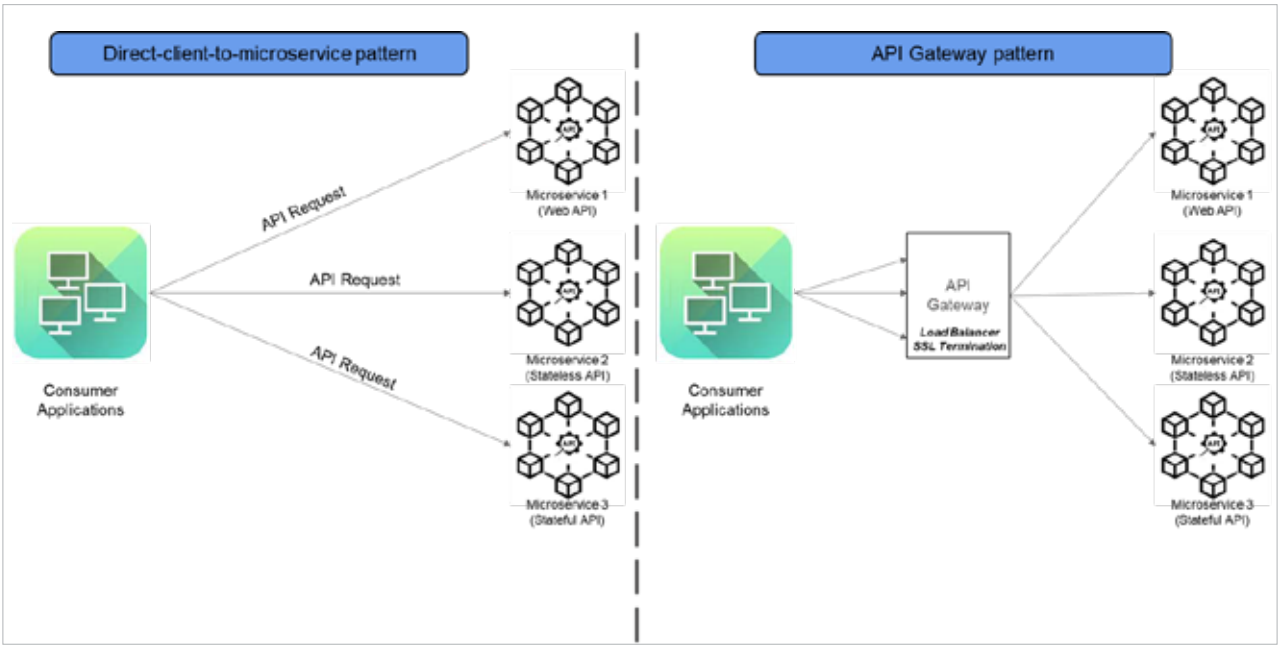

Figure 2: API gateway pattern

service design to understand the design principles and guidelines followed in the architecture.

Figure 3 shows an assessment which is based on a survey of questions about key software delivery outcomes, general architecture, inter-service communication, deployment and reliability, observability, externalised configuration, supporting infrastructure, libraries and frameworks, documentation, and organisation and process. Based on the responses for these questionnaires, the platform prepares a score for an application.

We can also view the impact on the score matrix to understand which section can have a high impact (e.g., general architecture or deployment) and which one can have a low impact (e.g., inter-service communication). Using this, we can decide if we must re-architect a solution for an improved microservices design.

We can double tap on each section to get details on the score to re-validate our response to the survey. This platform is a great tool to understand the agility of our microservices architecture.

Advantages of using microservices architecture

Scalability: Microservices allow individual components of an application to be scaled independently based on demand. This means that if one service experiences high traffic, it can be scaled without affecting the entire system.

Example: Netflix uses microservices to handle millions of users simultaneously, and can scale specific services like video streaming independently to manage varying loads.

Maintainability: Smaller, focused services are easier to understand, develop, and maintain. Changes can be made to one service without impacting others, reducing the risk of introducing bugs.

Example: Amazon employs microservices to manage its vast e-commerce platform, allowing teams to update services like payment processing or inventory management independently.

Deployment: Microservices enable continuous deployment and integration practices. Teams can deploy updates to individual services without needing to redeploy the entire application, leading to faster release cycles.

Example: Spotify uses microservices to deploy new features and updates frequently, allowing them to iterate quickly based on user feedback.

Tech diversity: Different microservices can be built using different technologies and programming languages, allowing teams to choose the best tools for each specific task.

Example: At eBay, various microservices are developed using different technologies, such as Java for backend services and Node.js for real-time applications, optimising performance and development efficiency.

Fault isolation: If one microservice fails, it does not necessarily bring down the entire system. This isolation helps in maintaining overall system stability and reliability.

Example: In a banking application, if the payment processing service fails, other services like account management can continue to function, ensuring that users can still access their accounts

Team autonomy: Microservices allow teams to work independently on different services, fostering a culture